

Day 8 and 9: Component Communication

Sections:-

- 1) Parent to Child Data Sharing
- 2) Child to Parent Data Sharing
- 3) Child-Child Data Sharing/ Services

1) Parent to Child Data Sharing:-

Component communication means being able to share data from one component to another. The first type of component communication is **Parent to Child Communication**.

Suppose you have an `AppComponent` acting as the parent and a component named `Child1Component` inside it acting as the child. You want to send data from the parent component to the child component.

To do this, Angular provides the `@Input` decorator.

Steps

- 1) Define the variable in the parent component

Create the variable that you want to send in the parent's `.ts` file.

Example:

```
parentData = "Hello from Parent";
```

2) Display the variable in the parent component

Print the variable in the parent's HTML file using interpolation. This helps you verify that the value displayed in the parent and child is the same.

Example:

```
<p>{{ parentData }}</p>
```

3) Create a variable in the child component using `@Input`

In the child's `.ts` file, create an empty variable and decorate it with `@Input()`.

This variable will receive the value sent by the parent.

Example:

```
@Input() childData: string = "";
```

4) Display the received value in the child component

Print the variable in the child's HTML file using interpolation.

Example:

```
<p>{{ childData }}</p>
```

5) Bind the parent's variable to the child's variable

While calling the child's selector inside the parent's HTML file, use property binding.

Example:

```
<app-child1 [childData]="parentData"></app-child1>
```

Here:

- `childData` is the variable inside the child component.
- `parentData` is the variable inside the parent component.

Angular automatically copies the value of `parentData` into `childData`.

2) Child to Parent Communication

Sometimes the child component needs to send data back to the parent component. This is called **Child to Parent Communication**. Suppose you have an `AppComponent` acting as the parent and a component named `Child2Component` acting as the child. You want to send data from `Child2Component` to `AppComponent`.

To do this, Angular provides the `@Output` decorator and `EventEmitter`.

Steps

1) Create a variable in the child component

Create a variable that will contain the data you want to send to the parent.

Example:

```
childData = "Hello from Child";
```

2) Create an EventEmitter in the child component

Create an `EventEmitter` and decorate it with `@Output()`.

This will act as the channel through which data is sent to the parent.

Example:

```
@Output() childEvent = new EventEmitter<string>();
```

3) Emit the data from the child component

Create a method that sends the data using the EventEmitter.

Example:

```
sendData() {  
  this.childEvent.emit(this.childData);  
}
```

4) Create a button in the child component

Call the method when the button is clicked.

Example:

```
<button (click)="sendData()">  
  Send Data to Parent  
</button>
```

5) Create a variable in the parent component

This variable will store the data received from the child.

Example:

```
receivedData = "";
```

6) Create a method in the parent component

This method will receive the data sent by the child.

Example:

```
catchChildData(data: string) {  
  this.receivedData = data;  
}
```

7) Listen for the event while calling the child component

When using the child's selector in the parent's HTML file, bind the event to the parent method.

Example:

```
<app-child2
```

```
(childEvent)="catchChildData($event)">
</app-child2>
```

Here:

- `childEvent` is the EventEmitter created in the child.
 - `$event` contains the data sent by the child.
 - `catchChildData()` is the method in the parent.
-

8) Display the received data in the parent component

Use interpolation to verify that the data has been received.

Example:

```
<p>{{ receivedData }}</p>
```

3) Child to Child Communication Using Service:-

Sometimes two components need to share data, but neither is the parent of the other.

For example:

App Component

└─ Child1

└─ Child2

Suppose **Child1** wants to send data to **Child2**.

Since they are sibling components, we cannot directly use **@Input** and **@Output** between them. Instead, we use a **Service**.

***What is a Service?**

A service is a TypeScript class that can store data and make it available to multiple components. You can think of it as a common storage area that multiple components can access.

***Create a Service**

Run: `ng g s shared`

Angular creates a file called: `shared.service.ts`

This service will act as the common place through which the two components communicate.

Steps

1) Create a variable inside the service

Open `shared.service.ts` and create a variable inside the service class.

Example:

```
myBill: number = 1000;
```

This variable will be shared between the components.

2) Inject the service into Child1

Import the service and add it to the constructor.

Example:

```
constructor(private shared: SharedService) {}
```

Now Child1 can access everything inside the service.

3) Update the service variable from Child1

Suppose Child1 wants to change the bill amount.

Example:

```
this.shared.myBill = 2000;
```

Now the value stored inside the service has become **2000**.

4) Inject the service into Child2

Import the same service and add it to the constructor.

Example:

```
constructor(private shared: SharedService) {}
```

Since both components are using the same service, Child2 can access the updated value.

5) Access the value in Child2

Create a variable and assign the service value to it.

Example:

```
billAmount: number = 0;
```

```
ngOnInit() {  
    this.billAmount = this.shared.myBill;  
}
```

Or you can directly use:

```
this.shared.myBill
```

wherever needed.

6) Display the value in Child2

Use interpolation in the HTML file.

Example:

```
<p>{{ billAmount }}</p>
```

If Child1 changes the value to 2000, Child2 will be able to read that value from the service.

THANK YOU.